

# CSCI 112

## Programming with C

### LECTURE: 3

J. L. Pach

# OUTLINE:

- Symbolic Name
- ASCII
- Operators and their rules
  - Casting
  - Unary, Binary, Ternary
  - Relational & logical operators
  - Statements & expressions
  - Block

# SYMBOLIC

Name

# ARRAYS

```
<type> symbolic_name[size];
```

- When you specify the size of an array in square brackets, it is created with that exact size.

```
<type> symbolic_name[] = {value1, value2, value3};
```

- If you omit the size but provide initial values, the compiler counts them and creates an array of that size.

```
<type> symbolic_name[size] = {value1, value2};
```

- If you specify the size but don't initialize all elements, the remaining ones will have indeterminate, unpredictable values.

# DO YOU REMEMBER?

```c
#include <stdio.h>

int main()
{
        char string[12] = "Hello world";
        printf("%s", string);
        return 0;
}
```

# EXAMPLE OF AN ARRAY

```
int main()
{
        char string0[12] = "Hello world";
        char string1[]  = "Hello world";
        char string2[12] = { 72, 101, 108 ,108, 111, 32, 87, 111, 114, 108, 100, 0 };
        char string3[]  = { 72, 101, 108 ,108, 111, 32, 87, 111, 114, 108, 100, 0 };
        char string4[12] = { 'H', 'e', 'l' , 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd', '\0' };
        char string5[]  = { 'H', 'e', 'l' , 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd', '\0' };
        char string6[12]  = { 'H', 101, 'l' , 108, 'o', ' ', 'w', 'o', 'r', 'l', 'd', 0 };
        printf("%s\n", string0);          printf("%s\n", string1);
        printf("%s\n", string2);          printf("%s\n", string3);
        printf("%s\n", string4);          printf("%s\n", string5);
        printf("%s\n", string6);          return 0;
}
```

```c
int main()
{
	char string0[12] = "Hello world";

	char string1[]  = "Hello world";

	char string2[12] = { 72, 101, 108 ,108, 111, 32, 87, 111, 114, 108, 100, 0 };

	char string3[]  = { 72, 101, 108 ,108, 111, 32, 87, 111, 114, 108, 100, 0 };

	char string4[12] = { 'H', 'e', 'l' , 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd', '\0' };

	char string5[]  = { 'H', 'e', 'l' , 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd', '\0' };

	char string6[12]  = { 'H', 101, 'l' , 108, 'o', ' ', 'w', 'o', 'r', 'l', 'd', 0 };

	printf("%s\n", string0);
	printf("%s\n", string1);
	printf("%s\n", string2);
	printf("%s\n", string3);
	printf("%s\n", string4);
	printf("%s\n", string5);
	printf("%s\n", string6);

	return 0;

}
```

# EXAMPLE OF AN ARRAY

- Text preceded by `#` is a preprocessor section, the first line gives access to standard input and output functions, this is a header.
- Next, we have the main function defined, which returns an integer value. The `{}` brackets start and end the body of the main function.
- The `printf` function displays the string `Hello world` on the console.

**Result**

```
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
```

# ARRAYS

```
<type> symbolic_name[size];
```

- When you specify the size of an array in square brackets, it is created with that exact size.

```
<type> symbolic_name[] = {value1, value2, value3};
```

- If you omit the size but provide initial values, the compiler counts them and creates an array of that size.

```
<type> symbolic_name[size] = {value1, value2};
```

- If you specify the size but don't initialize all elements, the remaining ones will have indeterminate, unpredictable values.

# QUESTION

## How do we know which letter goes with which number?

```
int main()
{
  char text[] = { 72, 101, 108 ,108, 111, 32, 87, 111, 114, 108, 100, 10, 13, 0 };
  printf("%s", text);
  return 0;
}
```

```
Result:
----------
Hello World
```

| Val | Char | Val | Char | Val | Char | Val | Char |
|---|---|---|---|---|---|---|---|
| 0 | NUL (null) | 32 | SPACE | 64 | @ | 96 | ` |
| 1 | SOH (start of heading) | 33 | ! | 65 | A | 97 | a |
| 2 | STX (start of text) | 34 | " | 66 | B | 98 | b |
| 3 | ETX (end of text) | 35 | # | 67 | C | 99 | c |
| 4 | EOT (end of transmission) | 36 | $ | 68 | D | 100 | d |
| 5 | ENQ (enquiry) | 37 | % | 69 | E | 101 | e |
| 6 | ACK (acknowledge) | 38 | & | 70 | F | 102 | f |
| 7 | BEL (bell) | 39 | ' | 71 | G | 103 | g |
| 8 | BS (backspace) | 40 | ( | 72 | H | 104 | h |
| 9 | TAB (horizontal tab) | 41 | ) | 73 | I | 105 | i |
| 10 | LF (NL line feed, new line) | 42 | * | 74 | J | 106 | j |
| 11 | VT (vertical tab) | 43 | + | 75 | K | 107 | k |
| 12 | FF (NP form feed, new page) | 44 | , | 76 | L | 108 | l |
| 13 | CR (carriage return) | 45 | - | 77 | M | 109 | m |
| 14 | SO (shift out) | 46 | . | 78 | N | 110 | n |
| 15 | SI (shift in) | 47 | / | 79 | O | 111 | o |
| 16 | DLE (data link escape) | 48 | 0 | 80 | P | 112 | p |
| 17 | DC1 (device control 1) | 49 | 1 | 81 | Q | 113 | q |
| 18 | DC2 (device control 2) | 50 | 2 | 82 | R | 114 | r |
| 19 | DC3 (device control 3) | 51 | 3 | 83 | S | 115 | s |
| 20 | DC4 (device control 4) | 52 | 4 | 84 | T | 116 | t |
| 21 | NAK (negative acknowledge) | 53 | 5 | 85 | U | 117 | u |
| 22 | SYN (synchronous idle) | 54 | 6 | 86 | V | 118 | v |
| 23 | ETB (end of trans. block) | 55 | 7 | 87 | W | 119 | w |
| 24 | CAN (cancel) | 56 | 8 | 88 | X | 120 | x |
| 25 | EM (end of medium) | 57 | 9 | 89 | Y | 121 | y |
| 26 | SUB (substitute) | 58 | : | 90 | Z | 122 | z |
| 27 | ESC (escape) | 59 | ; | 91 | [ | 123 | { |
| 28 | FS (file separator) | 60 | < | 92 | \ | 124 | \| |
| 29 | GS (group separator) | 61 | = | 93 | ] | 125 | } |
| 30 | RS (record separator) | 62 | > | 94 | ^ | 126 | ~ |
| 31 | US (unit separator) | 63 | ? | 95 | _ | 127 | DEL |

# ASCII

- American Standard Code for Information Interchange
- is the most common character encoding format for text data in computers and on the Internet. In standard ASCII-encoded data, there are unique values for 128 alphabetic, numeric or special additional characters and control codes.
- `\0` equals 0 (NULL)
- `\n` equals 10, 13 (n\ + \r)
- `\t` equals 11
- `White_Space` equals 32

# MULTI-DIMENSIONAL ARRAYS

- One dimension:

```
<type> symbolic_name[size];
```

- Two dimensions:

```
<type> symbolic_name[size1][size2];
```

- Three dimensions:

```
<type> symbolic_name[size1][size2][size3];
```

- etc.

# SYMBOLIC NAMES WILL BE USED IN:

- Variables
- Arrays
- Functions
- **Labels***: Symbolic names will be used to mark specific locations or points within the program code. These labels can be used for various purposes, such as control flow statements, data references, or error handling.

# LABELS

# EVERY LINE OF CODE IN C CAN HAVE ITS OWN LABEL.

```
int main()
{
label1:  int x = /* inline comment */ 5;
label2:
label3:  char string[12] = "Hello world"; /* comment behind the line */
label4:  printf("%s", string);
label5:  /* a comment
         composed of
         a few lines */
label7:
label8:    return 0;
}
```

# OPERATORS

## and their rules

```
struct Point { int x; int y; }; int main()
{struct Point point = {1,2}, *ppoint = &point; int arr[] = {1,2}; int x = 5, y =-6; int * z; float f = 3.0f; /*code*/}
```

| Priority | Operator | Description | Associativity | Example | Result |
|---|---|---|---|---|---|
| 1 | (), [] | Parentheses; Array subscript | Left-to-right | arr[0] * (x + y) | 1 |
| | . | Structure and union member access | | point.x | 1 |
| | -> | Structure and union member access through pointer | | ppoint->x | 1 |
| 2 | ++, -- | Prefix & postfix increment and decrement | Right-to-left | ++x; x--; x; | 6, 6, 5 |
| | +, -, !, ~ | (Unary) plus and minus; Logical NOT and bitwise NOT | | y =-y; y =+y; !x; ~x | 6, -6,0, -6 |
| | *, & , && | Indirection (dereference); Address-of; Address-of labels | | z = &x; *z; | 6422276; 5 |
| | (type), sizeof | Cast, Size-of | | (int)3.0f; sizeof(x); | 3, 4 |
| 3 | *, /, % | Multiplication, division, and remainder | Left-to-right | 6/2 % 2 | 1 |
| 4 | +, - | Addition and subtraction | | 1 + 2; 3 - 1 | 3, 2 |
| 5 | <<, >> | Bitwise left shift and right shift | | 4 << 1; 4 >> 2 | 8, 1 |
| 6 | <, <=, >, >= | For relational operators <, > and ≤, ≥ respectively | | x<y; x<=y; x>y; x>=y | 1, 1, 0 ,0 |
| 7 | ==, != | For relational = and ≠ respectively | | x == y, x != 1 | 1, 0 |
| 8 | & | Bitwise AND | | 7 & 3 | 3 |
| 9 | ^ | Bitwise XOR (exclusive or) | | 255 ^ 0 | 255 |
| 10 | \| | Bitwise OR (inclusive or) | | 7 \| 3 | 7 |
| 11 | && | Logical AND | | 1 && 0 | 0 |
| 12 | \|\| | Logical OR | | 1 \|\| 0 | 1 |
| 13 | ?: | Ternary conditional | Right-to-left | x = (x > y) ? y : x; | -6 |
| 14 | = | Simple assignment | | x = y; | -6 |
| | +=, -=, *=, /=, %= | Assignment by sum, difference, product, quotient, remainder | | x+=1; x-=1; //etc. | 6, 5 |
| | <<=, >>=, &=, ^=, \|= | Assignment by bitwise left shift, right shift, AND, XOR, OR | | 3<<=1, 8>>=2 //etc. | 6, 2 |
| 15 | , | Comma | Left-to-right | x = 3, y = 1; | 3 |

# CASTING

```
struct Point { int x; int y; }; int main()
{struct Point point = {1,2}, *ppoint = &point; int arr[] = {1,2}; int x = 5, y =-6; int * z; float f = 3.0f; /*code*/}
```

| Priority | Operator | Description | Associativity | Example | Result |
|---|---|---|---|---|---|
| 1 | (), [] | Parentheses; Array subscript | Left-to-right | arr[0] * (x + y) | 1 |
| | . | Structure and union member access | | point.x | 1 |
| | -> | Structure and union member access through pointer | | ppoint->x | 1 |
| 2 | ++, -- | Prefix & postfix increment and decrement | Right-to-left | ++x; x--; x; | 6, 6, 5 |
| | +, -, !, ~ | (Unary) plus and minus; Logical NOT and bitwise NOT | | y =-y; y =+y; !x; ~x | 6, -6,0, -6 |
| | *, & , && | Indirection (dereference); Address-of; Address-of labels | | z = &x; *z; | 6422276; 5 |
| | **(type), sizeof** | **Cast, Size-of** | | **(int)3.0f; sizeof(x);** | **3, 4** |
| 3 | *, /, % | Multiplication, division, and remainder | Left-to-right | 6/2 % 2 | 1 |
| 4 | +, - | Addition and subtraction | | 1 + 2; 3 - 1 | 3, 2 |
| 5 | <<, >> | Bitwise left shift and right shift | | 4 << 1; 4 >> 2 | 8, 1 |
| 6 | <, <=, >, >= | For relational operators <, > and ≤, ≥ respectively | | x<y; x<=y; x>y; x>=y | 1, 1, 0 ,0 |
| 7 | ==, != | For relational = and ≠ respectively | | x == y, x != 1 | 1, 0 |
| 8 | & | Bitwise AND | | 7 & 3 | 3 |
| 9 | ^ | Bitwise XOR (exclusive or) | | 255 ^ 0 | 255 |
| 10 | \| | Bitwise OR (inclusive or) | | 7 \| 3 | 7 |
| 11 | && | Logical AND | | 1 && 0 | 0 |
| 12 | \|\| | Logical OR | | 1 \|\| 0 | 1 |
| 13 | ?: | Ternary conditional | Right-to-left | x = (x > y) ? y : x; | -6 |
| 14 | = | Simple assignment | | x = y; | -6 |
| | +=, -=, *=, /=, %= | Assignment by sum, difference, product, quotient, remainder | | x+=1; x-=1; //etc. | 6, 5 |
| | <<=, >>=, &=, ^=, \|= | Assignment by bitwise left shift, right shift, AND, XOR, OR | | 3<<=1, 8>>=2 //etc. | 6, 2 |
| 15 | , | Comma | Left-to-right | x = 3, y = 1; | 3 |

# CASTING

```
1   int   main()
2   {              5
3         int      a = 5;
4         float    b = 8.2f ;   8.199999 81
5                                          // The expression 'b + a' promotes 'a' to float automatically,
6         int      c = b + a;      13      // so     the addition is done in floating          - point.
7                                          //The float result is implicitly converted back to int (truncation).
8         float    d = b + a;      13.19999 98
9         return    0;
10  }
```

Although the `int` and `float` types in C are the same length, a much larger number can be stored in a `float` type than in an `int`. This is due to a different way of encoding the bits. As a result, when it comes to casting, `float` **is treated as a larger type** than `int`. Therefore, if an int appears in an expression with a `float`, the int is always first cast to `float`, and only then is the result computed.

# WHAT IS CASTING?

The process of converting a value of one data type to another.

There are two types of casting:

- Explicit
- Implicit

The purpose:

- To perform arithmetic operations on different data types.
- To assign a value of one type to a variable of another type.
- To access specific parts of a data structure.

# EXPLICIT CASTING

Manual conversion performed by the programmer using a cast operator.

```
int   a = 5;          // 5
float b = 8.2f;             // 8.199999_81 <--
c = (int)b + a;             // 13
float d = b + (float)a;     // 13.19999_98 <--
```

If we put a data type in parentheses before a symbolic name (variable), we perform a type cast of that variable to the specified type. When casting a `float` to an `int`, the number is **truncated**, which can **lead to loss of information**. In the opposite case, casting to a **wider** type is **always safe and does not cause data loss**, but casting to a narrower type may result in loss of information.

# IMPLICIT CASTING

- Automatic conversion performed by the compiler.
- Automatic casting will always cast to a wider type to never lose information.
- There is no automatic conversion from float to double, this is the only exception.
- Even though expressions in the return statement are always implicitly cast to the function's return type, it is considered good practice to explicitly cast the value to emphasize that this is intentional. If the programmer doesn't perform this explicit cast, the compiler will do it for them, but this is generally considered poor programming style...
- When occurs:
  - When assigning a value of a smaller type to a larger one (e.g. int to float).
  - In arithmetic expressions where different types are mixed.

# WHAT'S THE DIFFERENCE?

```
int   a = 5;              // 5
float b = 8.2f;           // 8.199999_81 <--
int   c = b + a;          // 13
float d = b + a;          // 13.19999_98 <--
```

```
int   a = 5;              // 5
float b = 8.2f;           // 8.199999_81 <--
c = (int)b + a;           // 13
float d = b + (float)a;   // 13.19999_98 <--
```

# Unary, Binary, Ternary

# OPERATORS

- Operators specify what is to be done to variables (also pointers or labels)
- Operators can be:
  - **unary** (e. g. `-`, `++`), **binary** (e. g. `+`, `-`, `*`, `/`), **ternary** (`?:`)

```
int main()
{
  int i = 1;
  i = -i;              /* unary -           */
  i++;                 /* unary ++                */
  i = i + 1; i = i - 1;       /* binary +, -             */
  i = i * 1; i = i / 1;       /* binary *,               */
  i ? (i > 0) : 1 ; 0;        /* ternary conditional   */
}
```

# BASIC OPERATORS

```
struct Point { int x; int y; }; int main()
{struct Point point = {1,2}, *ppoint = &point; int arr[] = {1,2}; int x = 5, y =-6; int * z; float f = 3.0f; /*code*/}
```

| Priority | Operator | Description | Associativity | Example | Result |
|---|---|---|---|---|---|
| **1** | **(), []** | **Parentheses; Array subscript** | **Left-to-right** | **arr[0] * (x + y)** | **1** |
| | . | Structure and union member access | | point.x | **1** |
| | -> | Structure and union member access through pointer | | ppoint->x | **1** |
| **2** | ++, -- | Prefix & postfix increment and decrement | Right-to-left | ++x; x--; x; | **6, 6, 5** |
| | +, -, !, ~ | (Unary) plus and minus; Logical NOT and bitwise NOT | | y =-y; y =+y; !x; ~x | **6, -6,0, -6** |
| | *, & , && | Indirection (dereference); Address-of; Address-of labels | | z = &x; *z; | **6422276; 5** |
| | (type), sizeof | Cast, Size-of | | (int)3.0f; sizeof(x); | **3, 4** |
| **3** | ***, /, %** | **Multiplication, division, and remainder** | **Left-to-right** | **6/2 % 2** | **1** |
| **4** | **+, -** | **Addition and subtraction** | | **1 + 2; 3 - 1** | **3, 2** |
| **5** | <<, >> | Bitwise left shift and right shift | | 4 << 1; 4 >> 2 | **8, 1** |
| **6** | <, <=, >, >= | For relational operators <, > and ≤, ≥ respectively | | x<y; x<=y; x>y; x>=y | **1, 1, 0 ,0** |
| **7** | ==, != | For relational = and ≠ respectively | | x == y, x != 1 | **1, 0** |
| **8** | & | Bitwise AND | | 7 & 3 | **3** |
| **9** | ^ | Bitwise XOR (exclusive or) | | 255 ^ 0 | **255** |
| **10** | \| | Bitwise OR (inclusive or) | | 7 \| 3 | **7** |
| **11** | && | Logical AND | | 1 && 0 | **0** |
| **12** | \|\| | Logical OR | | 1 \|\| 0 | **1** |
| **13** | ?: | Ternary conditional | Right-to-left | x = (x > y) ? y : x; | **-6** |
| **14** | = | Simple assignment | | x = y; | **-6** |
| | +=, -=, *=, /=, %= | Assignment by sum, difference, product, quotient, remainder | | x+=1; x-=1; //etc. | **6, 5** |
| | <<=, >>=, &=, ^=, \|= | Assignment by bitwise left shift, right shift, AND, XOR, OR | | 3<<=1, 8>>=2 //etc. | **6, 2** |
| **15** | , | Comma | Left-to-right | x = 3, y = 1; | **3** |

# BASICS OF MATHEMATICS 1/2

- From basic mathematics, it is known that multiplication has higher precedence than addition, and this precedence can be changed using parentheses. These rules work exactly the same way in the C language as in mathematics.
- However, because the representation of integers differs from floating-point numbers, as mentioned earlier, when a floating-point number is involved in an operation with an integer, the integer is implicitly cast to a floating-point number

# BASICS OF MATHEMATICS 2/2

- Another important point is the division operator `/`. When used with integers, it performs integer division, meaning that `5 / 3` results in `1`. If at least one operand is a floating-point number, the division produces a floating-point result, so `5.0 / 3` evaluates to approximately `1.6666`.
- Finally, there is the modulo operator `%`, which returns the **remainder of integer division**. For example, `5 % 2` equals `1`.

# EXAMPLE - DIVISION 1/2

```c
/* Declarations */
int x = 5, y = 3;
float a = 5.0, b = 3.0;

printf("%s\n", "Integers:");    // Integers:
printf("x = %d\n", x);          // x = 5
printf("y = %d\n", y);          // y = 3

printf("%s\n","Floats:");               // Floats:
printf("a = %f\n", a);          // a = 5.000000
printf("b = %f\n", b);          // b = 3.000000

/* integer division */
printf("Integer / Integer:\n"); // Integer / Integer:
printf("x / y = %d\n", x / y);  // x / y = 1

printf("\nInteger / Float:\n"); // Integer / Float:        /* x promoted to float */
printf("x / b = %f\n", x / b);  // x / b = 1.666667        /* incorrect, prints garbage */
printf("x / b with %%d = %d (wrong!)\n", x / b);            /* x / b with %d = -1431655765 (wrong!) */
```

# EXAMPLE - DIVISION 2/2

```
/* Declarations */
int x = 5, y = 3;
float a = 5.0, b = 3.0;

printf("\nFloat / Integer:\n"); // Float / Integer:          /* y promoted to float */
printf("a / y = %f\n", a / y);   // a / y = 1.666667

/* float division */
printf("\nFloat / Float:\n");    //Float / Float:
printf("a / b = %f\n", a / b);   // a / b = 1.666667        /*normal float division*/
```

# ++ , -- , =

**Increment, decrement, statement operator**

```
struct Point { int x; int y; }; int main()
{struct Point point = {1,2}, *ppoint = &point; int arr[] = {1,2}; int x = 5, y =-6; int * z; float f = 3.0f; /*code*/}
```

| Priority | Operator | Description | Associativity | Example | Result |
|---|---|---|---|---|---|
| 1 | (), [] | Parentheses; Array subscript | Left-to-right | arr[0] * (x + y) | 1 |
| | . | Structure and union member access | | point.x | 1 |
| | -> | Structure and union member access through pointer | | ppoint->x | 1 |
| **2** | **++, --** | **Prefix & postfix increment and decrement** | **Right-to-left** | **++x; x--; x;** | **6, 6, 5** |
| | +, -, !, ~ | (Unary) plus and minus; Logical NOT and bitwise NOT | | y =-y; y =+y; !x; ~x | 6, -6,0, -6 |
| | *, & , && | Indirection (dereference); Address-of; Address-of labels | | z = &x; *z; | 6422276; 5 |
| | (type), sizeof | Cast, Size-of | | (int)3.0f; sizeof(x); | 3, 4 |
| 3 | *, /, % | Multiplication, division, and remainder | Left-to-right | 6/2 % 2 | 1 |
| 4 | +, - | Addition and subtraction | | 1 + 2; 3 - 1 | 3, 2 |
| 5 | <<, >> | Bitwise left shift and right shift | | 4 << 1; 4 >> 2 | 8, 1 |
| 6 | <, <=, >, >= | For relational operators <, > and ≤, ≥ respectively | | x<y; x<=y; x>y; x>=y | 1, 1, 0 ,0 |
| 7 | ==, != | For relational = and ≠ respectively | | x == y, x != 1 | 1, 0 |
| 8 | & | Bitwise AND | | 7 & 3 | 3 |
| 9 | ^ | Bitwise XOR (exclusive or) | | 255 ^ 0 | 255 |
| 10 | \| | Bitwise OR (inclusive or) | | 7 \| 3 | 7 |
| 11 | && | Logical AND | | 1 && 0 | 0 |
| 12 | \|\| | Logical OR | | 1 \|\| 0 | 1 |
| 13 | ?: | Ternary conditional | Right-to-left | x = (x > y) ? y : x; | -6 |
| **14** | **=** | **Simple assignment** | | **x = y;** | **-6** |
| | **+=, -=, *=, /=, %=** | **Assignment by sum, difference, product, quotient, remainder** | | **x+=1; x-=1; //etc.** | **6, 5** |
| | **<<=, >>=, &=, ^=, \|=** | **Assignment by bitwise left shift, right shift, AND, XOR, OR** | | **3<<=1, 8>>=2 //etc.** | **6, 2** |
| 15 | , | Comma | Left-to-right | x = 3, y = 1; | 3 |

# ++, --, =, ETC 1/2

```
int main()
{
  int i = 1, j = 2;
  j++;          // j = j + 1;

  i = j++;      // i = j;
                // j = j + 1;

  i = ++j;      // j = j + 1;
                // i = j;

  i += j;       // i = i + j;

  i = j = 2 + j;       // j = 2 + j;
                // i = j;
}
```

# `++`,`--`,`=`, ETC 2/2

- Prefix increment `++` and decrement `--` operators increase (decrease) the operand's value first, then use the new value.
- Postfix increment `++` and decrement `--` operators use the operand's current value first, then increase (decrease) the operand's value.
- Assignment by sum refers to the process of assigning a value to a variable using the addition operator + or the addition assignment operator `+=` to combine the assigned value with the existing value of the variable.

# RELATIONAL

## & logical operators

```
struct Point { int x; int y; }; int main()
{struct Point point = {1,2}, *ppoint = &point; int arr[] = {1,2}; int x = 5, y =-6; int * z; float f = 3.0f; /*code*/}
```

| Priority | Operator | Description | Associativity | Example | Result |
|---|---|---|---|---|---|
| 1 | (), [] | Parentheses; Array subscript | Left-to-right | arr[0] * (x + y) | 1 |
| | . | Structure and union member access | | point.x | 1 |
| | -> | Structure and union member access through pointer | | ppoint->x | 1 |
| 2 | ++, -- | Prefix & postfix increment and decrement | Right-to-left | ++x; x--; x; | 6, 6, 5 |
| | +, -, !, ~ | (Unary) plus and minus; Logical NOT and bitwise NOT | | y =-y; y =+y; !x; ~x | 6, -6,0, -6 |
| | *, & , && | Indirection (dereference); Address-of; Address-of labels | | z = &x; *z; | 6422276; 5 |
| | (type), sizeof | Cast, Size-of | | (int)3.0f; sizeof(x); | 3, 4 |
| 3 | *, /, % | Multiplication, division, and remainder | Left-to-right | 6/2 % 2 | 1 |
| 4 | +, - | Addition and subtraction | | 1 + 2; 3 - 1 | 3, 2 |
| 5 | <<, >> | Bitwise left shift and right shift | | 4 << 1; 4 >> 2 | 8, 1 |
| 6 | **<, <=, >, >=** | **For relational operators <, > and ≤, ≥ respectively** | | **x<y; x<=y; x>y; x>=y** | **1, 1, 0 ,0** |
| 7 | **==, !=** | **For relational = and ≠ respectively** | | **x == y, x != 1** | **1, 0** |
| 8 | & | Bitwise AND | | 7 & 3 | 3 |
| 9 | ^ | Bitwise XOR (exclusive or) | | 255 ^ 0 | 255 |
| 10 | \| | Bitwise OR (inclusive or) | | 7 \| 3 | 7 |
| 11 | **&&** | **Logical AND** | | **1 && 0** | 0 |
| 12 | **\|\|** | **Logical OR** | | **1 \|\| 0** | 1 |
| 13 | ?: | Ternary conditional | Right-to-left | x = (x > y) ? y : x; | -6 |
| 14 | = | Simple assignment | | x = y; | -6 |
| | +=, -=, *=, /=, %= | Assignment by sum, difference, product, quotient, remainder | | x+=1; x-=1; //etc. | 6, 5 |
| | <<=, >>=, &=, ^=, \|= | Assignment by bitwise left shift, right shift, AND, XOR, OR | | 3<<=1, 8>>=2 //etc. | 6, 2 |
| 15 | , | Comma | Left-to-right | x = 3, y = 1; | 3 |

# RELATIONAL & LOGICAL OPERATORS

`<`, `<=`, `>`, `>=`, `==`, `!=`, `&&`, `||`

- Relational and logical operators return 1 for a `true` condition and 0 for a `false` condition. The result type of these operators is `int`.
- In if and `while` statements, the C language considers the logical value of the conditional expression. This means that any value other than `0` (**including positive, negative, characters, pointers, etc.**) will be treated as `true`, while the value `0` will be treated as `false`.
- The `bool` type is not a built-in type because it is not essential for basic programming operations. However, to use the `bool` type, we need to include the `<stdbool.h>` header file.

# RELATIONAL & LOGICAL OPERATORS - EXAMPLE

```
#include <stdio.h>
#include <stdbool.h>
int main()
{
  bool is_true  = true;
  bool is_false = false;
  bool result = is_true && is_false;
  printf("%d\n", result);
  result = 3 > 1;
  printf("%d\n", result);
}
```

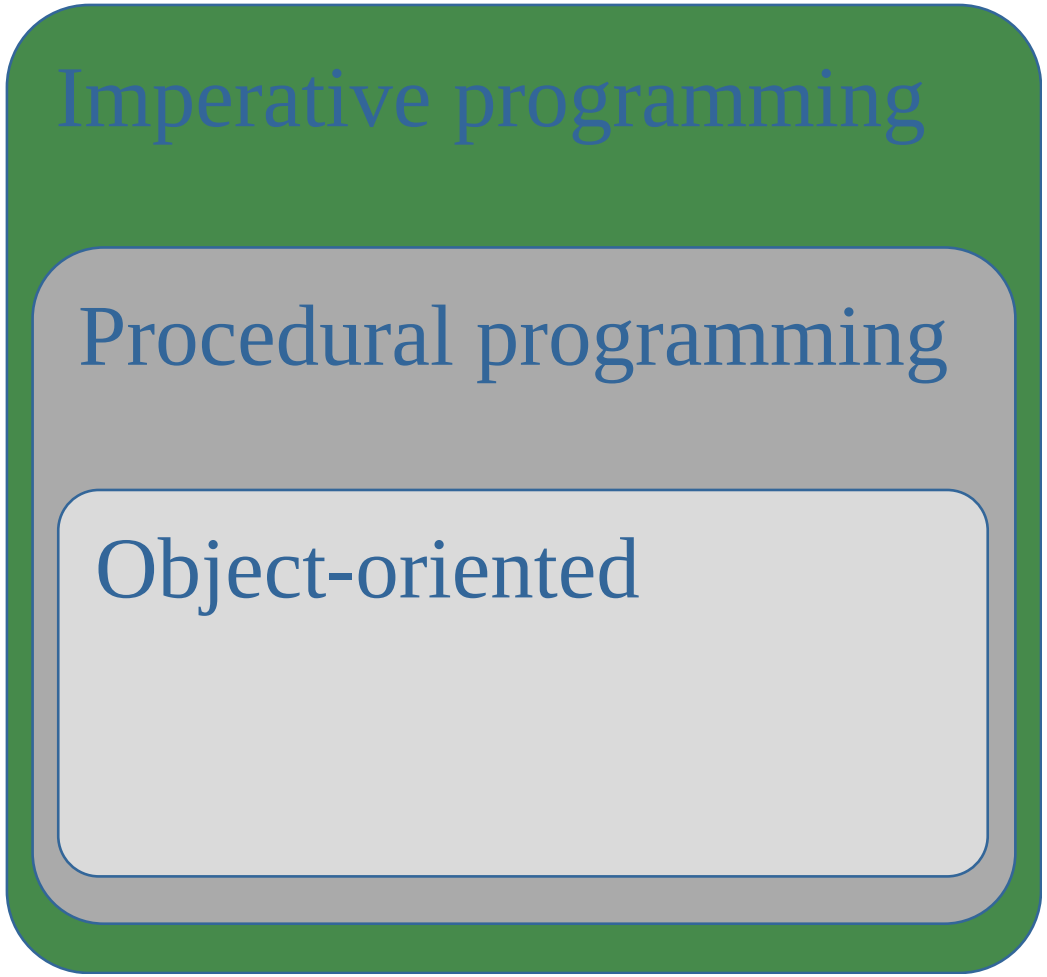
```
Result:
0
1
```

# STATEMENTS

## & expressions

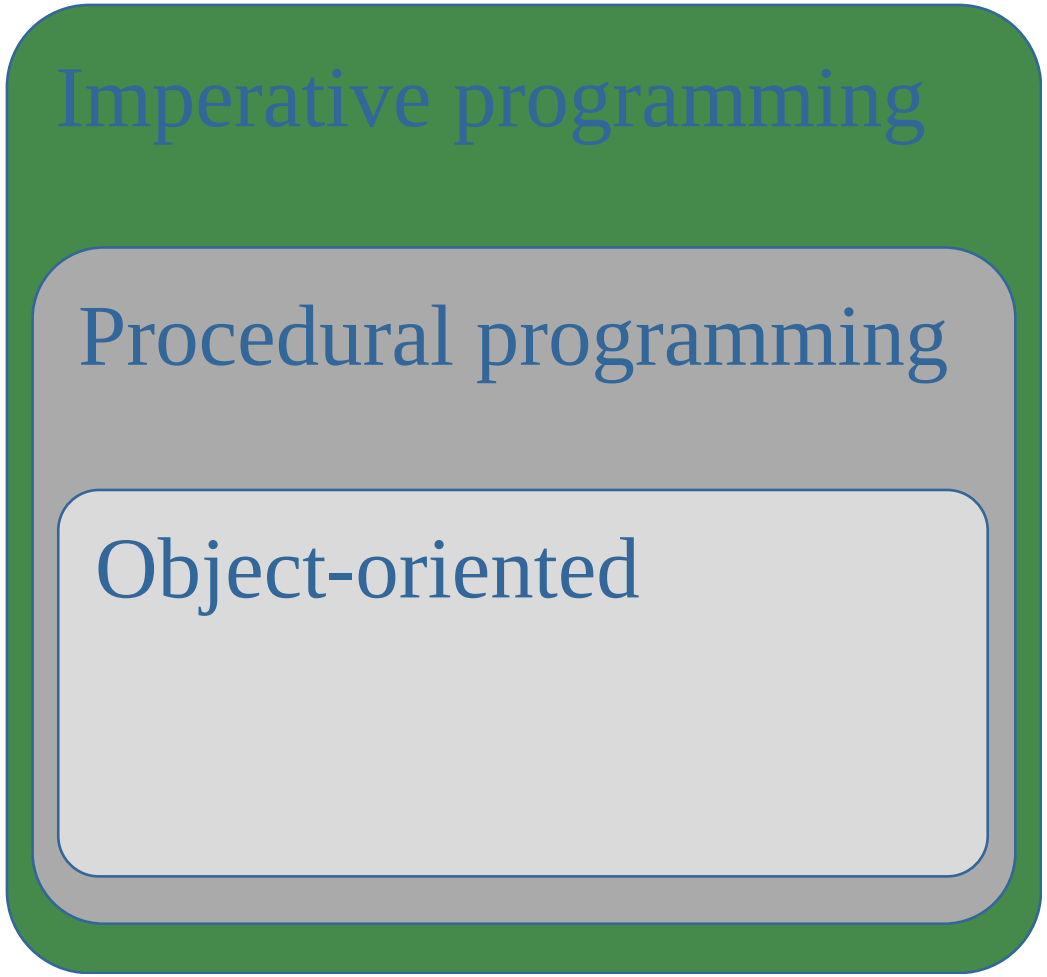
# IMPERATIVE PROGRAMMING

- **Imperative programming** is a programming paradigm that uses s**tatements to change a program's state**.
- It is like giving the computer instructions step by step on what to do.



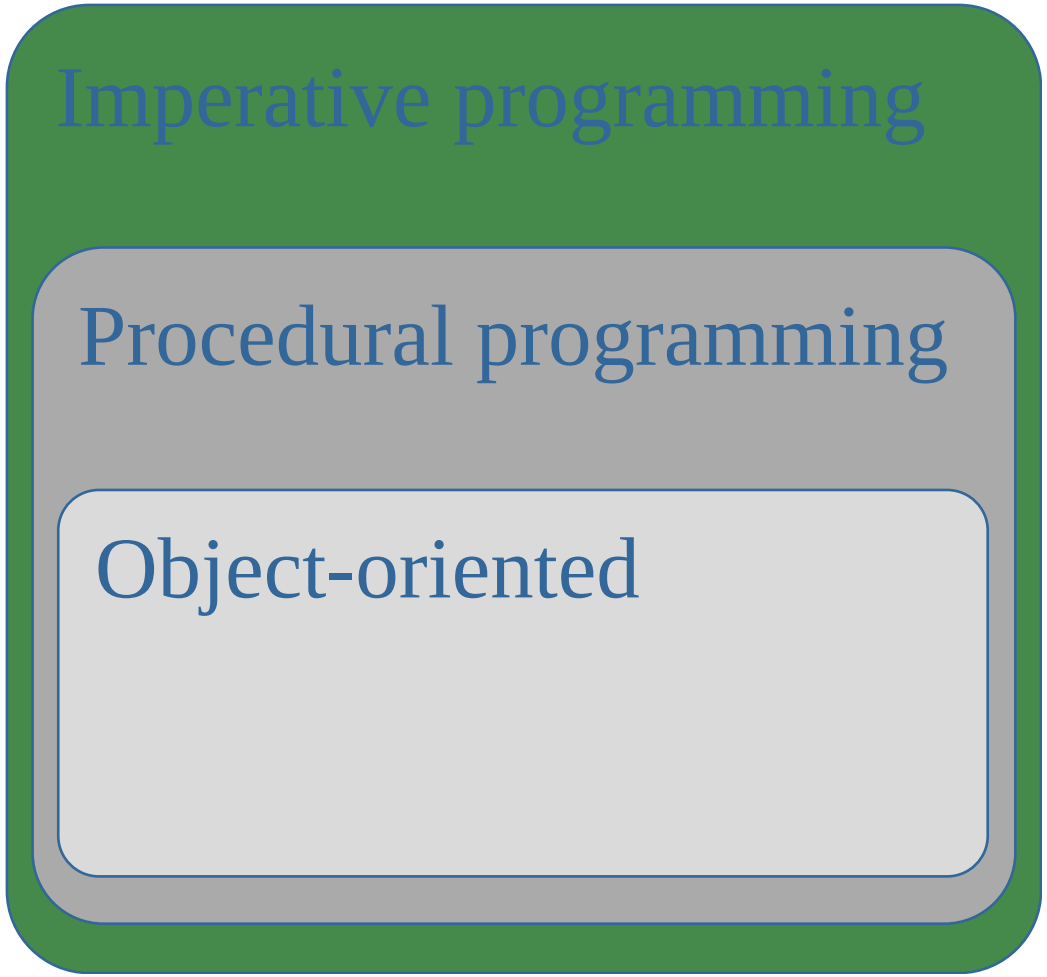
# PROCEDURAL PROGRAMMING

- **Procedural programming** is a type of imperative programming that breaks down a program into smaller, independent procedures (functions).
- This makes the program more organized and easier to maintain.



# OBJECT-ORIENTED PROGRAMMING

- **Object-oriented programming** further extends the concepts of procedural programming by organizing code into objects that encapsulate data and behavior.



# STATEMENT & EXPRESSION

- In computer science, a **statement** is a complete instruction that can modify the state of a program, such as assigning a value to a variable, calling a function, or jumping to a different part of the code. **Statements do not produce a value on their own.**
- An **expression**, on the other hand, is a combination of variables, constants, functions, and operators that evaluates to a single value (**The absence of a returned value from a function is also a result**). Expressions can be used within statements to provide values for operations or to control the flow of the program. However, **expressions themselves do not directly modify the state of the program.**

# NULL / EMPTY STATEMENT;

```
int main()
{
        ;                               /*  Empty statement    */
        printf("Hello world!\n");                   /*  Second statement   */
        printf("Hello"); printf(" world!\n");               /*  3rd & 4th statement in one line */
}
```

- A single semicolon `;` in C language is a statement called a `null` statement or empty statement.
- A semicolons `;` are used to terminate statements

# CONCLUSIONS

- *In C, a code line is not the same as a statement*
- *An entire program can be written in a single line*
- *As the compiler separates statements with semicolons* `;` *and not newlines*
- *Therefore, a single line of text (with a newline character) can contain multiple statements.*

# BLOCK

# BLOCK & COMPLEX(COMPOUND) STATEMENTS

- **Block** can be:
  - substitute for simple statement,
  - defined within a block (Nested blocks),
  - empty `{}`.
- Variables can be declared inside
- Variables declared inside a block with the same name hide access to variables outside that block
- No semicolon at end of block
- After a code block ends, all variables declared within that block cease to exist and are no longer accessible in the rest of the program.

# BLOCK & COMPLEX(COMPOUND) STATEMENTS - EXAMPLE

```
int main()
{
    char* text = "Hello world\n";
    printf(text);                          // Hello world
    {
      char* text = "Bye world\n";
      printf(text);                        // Bye world
      {
        char* text = "Aloha world\n";
        printf(text);                      // Aloha world
      }
    }
    printf(text);                          // Hello world
}
```

# SUMMARY

- A **statement** can modify the state of a program, such as assigning a value to a variable, calling a function, or jumping to a different part of the code.
- Statements do not produce a value on their own.
- An **expression** is a combination of variables, constants, functions, and operators that evaluates to a single value.
- Expressions themselves do not directly modify the state of the program.

# EXAMPLE

```
int main()
{
        int x, y, _;
        _ = x = y = 5;

        _ = x = y = 5;
        _ = x = (5);
        _ = x = 5;
        _ = (5);
        _ = 5;
        (5)
}
```

# CONSEQUENCES

- In the C language, **every** assignment after assigning a value to a variable (pointer, label), since it is also an expression, will generate a value again, which will be ignored by the compiler at the end.
- We can use the underscore `_` discard in C, but you need to declare the variable beforehand. It's a great mechanism that improves the readability of the programmer's intent.

# THANK

You